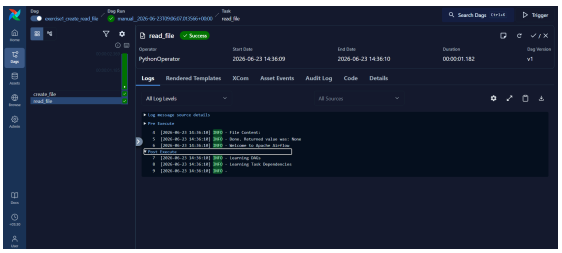
Simple auth manager | Password for user 'admin': v9qqxhCh3dQhdHTU

ex01.py

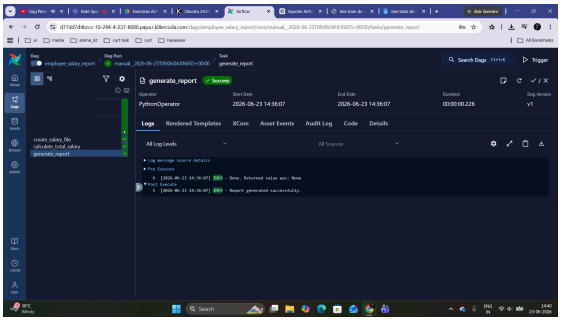
```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime

# Task 1: Create file
def create_file():
    with open('/tmp/message.txt', 'w') as f:
        f.write("Welcome to Apache Airflow\n")
        f.write("Learning DAGs\n")
        f.write("Learning Task Dependencies\n")

    print("File created successfully.")

# Task 2: Read file
def read_file():
    with open('/tmp/message.txt', 'r') as f:
        content = f.read()

    print("File Content:")
    print(content)

with DAG(
    dag_id='exercise1_create_read_file',
    start_date=datetime(2026, 6, 23),
    schedule=None,
    catchup=False
) as dag:

    create_file_task = PythonOperator(
        task_id='create_file',
        python_callable=create_file
    )

    read_file_task = PythonOperator(
        task_id='read_file',
        python_callable=read_file
    )

    # Task Dependency
    create_file_task >> read_file_task
```



ex02.py

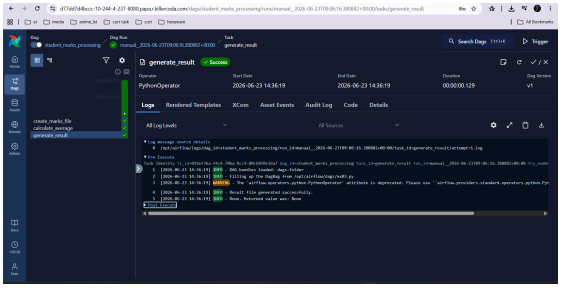
```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime

# Task 1: Create employee file
def create_salary_file():
    with open('/tmp/employees.txt', 'w') as f:
        f.write("Rahul,45000\n")
        f.write("Priya,52000\n")
        f.write("Amit,61000\n")
        f.write("Sneha,48000\n")

    print("Employee file created successfully.")

# Task 2: Calculate total salary
def calculate_total_salary(ti):
    total_salary = 0

    with open('/tmp/employees.txt', 'r') as f:
        for line in f:
            name, salary = line.strip().split(',')
            total_salary += int(salary)

    print(f"Total Salary = {total_salary}")

    # Store value for next task
    ti.xcom_push(key='total_salary', value=total_salary)
    ti.xcom_push(key='employee_count', value=4)

# Task 3: Generate report
def generate_report(ti):
    total_salary = ti.xcom_pull(
        task_ids='calculate_total_salary',
        key='total_salary'
    )

    employee_count = ti.xcom_pull(
        task_ids='calculate_total_salary',
        key='employee_count'
    )

    with open('/tmp/report.txt', 'w') as f:
        f.write("Salary Report\n")
        f.write(f"Employees = {employee_count}\n")
        f.write(f"Total Salary = {total_salary}\n")

    print("Report generated successfully.")

with DAG(
    dag_id='employee_salary_report',
    start_date=datetime(2026, 6, 23),
    schedule=None,
```

```
    catchup=False
) as dag:

    create_salary_file_task = PythonOperator(
        task_id='create_salary_file',
        python_callable=create_salary_file
    )

    calculate_total_salary_task = PythonOperator(
        task_id='calculate_total_salary',
        python_callable=calculate_total_salary
    )

    generate_report_task = PythonOperator(
        task_id='generate_report',
        python_callable=generate_report
    )

    # Task Dependencies
    create_salary_file_task >> \
    calculate_total_salary_task >> \
    generate_report_task
```



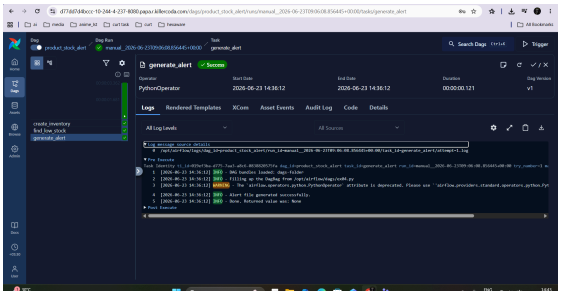
[ex03.py](#)

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime

# Task 1: Create marks file
def create_marks_file():
    with open('/tmp/marks.txt', 'w') as f:
        f.write("Math,80\n")
        f.write("Science,75\n")
        f.write("English,90\n")
        f.write("Python,95\n")

    print("Marks file created successfully.")

# Task 2: Calculate average marks
def calculate_average(ti):
    total_marks = 0
    subject_count = 0

    with open('/tmp/marks.txt', 'r') as f:
        for line in f:
            subject, marks = line.strip().split(',')
            total_marks += int(marks)
            subject_count += 1

    average = total_marks / subject_count

    print(f"Average Marks = {average}")

    # Pass average to next task
    ti.xcom_push(key='average', value=average)

# Task 3: Generate result file
def generate_result(ti):
    average = ti.xcom_pull(
        task_ids='calculate_average',
        key='average'
    )

    result = "PASS" if average >= 50 else "FAIL"

    with open('/tmp/result.txt', 'w') as f:
        f.write(f"Average Marks = {average}\n")
        f.write(f"Result = {result}\n")

    print("Result file generated successfully.")

with DAG(
    dag_id='student_marks_processing',
    start_date=datetime(2026, 6, 23),
    schedule=None,
    catchup=False
) as dag:

    create_marks_file_task = PythonOperator(
        task_id='create_marks_file',
        python_callable=create_marks_file
    )

    calculate_average_task = PythonOperator(
        task_id='calculate_average',
        python_callable=calculate_average
    )

    generate_result_task = PythonOperator(
        task_id='generate_result',
        python_callable=generate_result
    )

    # Task Dependencies
    create_marks_file_task >> \
    calculate_average_task >> \
    generate_result_task
```



ex04.py

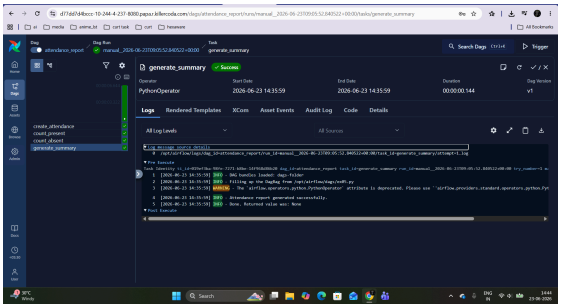
```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime

# Task 1: Create inventory file
def create_inventory():
    with open('/tmp/inventory.txt', 'w') as f:
        f.write("Rice,50\n")
        f.write("Oil,7\n")
        f.write("Soap,35\n")
        f.write("Sugar,10\n")
        f.write("Tea,5\n")

    print("Inventory file created successfully.")

# Task 2: Find low stock items
def find_low_stock(ti):
    low_stock_items = []

    with open('/tmp/inventory.txt', 'r') as f:
        for line in f:
            product, stock = line.strip().split(",")

            if int(stock) < 15:
                low_stock_items.append(product)

    print("Low Stock Items:")
    for item in low_stock_items:
        print(item)

    # Pass list to next task
    ti.xcom_push(key='low_stock_items',
value=low_stock_items)

# Task 3: Generate alert file
def generate_alert(ti):
    low_stock_items = ti.xcom_pull(
        task_ids='find_low_stock',
        key='low_stock_items'
    )

    with open('/tmp/alerts.txt', 'w') as f:
        f.write("Low Stock Alert\n")
        for item in low_stock_items:
            f.write(f"{item}\n")

    print("Alert file generated successfully.")

with DAG(
    dag_id='product_stock_alert',
    start_date=datetime(2026, 6, 23),
    schedule=None,
    catchup=False
) as dag:

    create_inventory_task = PythonOperator(
        task_id='create_inventory',
        python_callable=create_inventory
    )

    find_low_stock_task = PythonOperator(
        task_id='find_low_stock',
        python_callable=find_low_stock
    )

    generate_alert_task = PythonOperator(
        task_id='generate_alert',
        python_callable=generate_alert
    )

    # Task Dependencies
    create_inventory_task >> \
    find_low_stock_task >> \
    generate_alert_task
```



ex05.py

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime

# Task 1: Create attendance file
def create_attendance():
    with open('/tmp/attendance.txt', 'w') as f:
        f.write("Rahul,Present\n")
        f.write("Priya,Present\n")
        f.write("Amit,Absent\n")
        f.write("Sneha,Present\n")
        f.write("Kiran,Absent\n")

    print("Attendance file created successfully.")
```

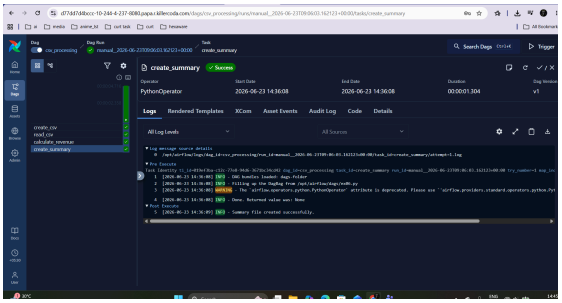
```
# Task 2: Count present students
def count_present(ti):
    present_count = 0

    with open('/tmp/attendance.txt', 'r') as f:
        for line in f:
            name, status = line.strip().split(',')

            if status == "Present":
                present_count += 1

    print(f"Present = {present_count}")

    ti.xcom_push(key='present_count',
value=present_count)

# Task 3: Count absent students
def count_absent(ti):
    absent_count = 0

    with open('/tmp/attendance.txt', 'r') as f:
        for line in f:
            name, status = line.strip().split(',')

            if status == "Absent":
                absent_count += 1

    print(f"Absent = {absent_count}")

    ti.xcom_push(key='absent_count',
value=absent_count)

# Task 4: Generate summary report
def generate_summary(ti):
    present_count = ti.xcom_pull(
        task_ids='count_present',
        key='present_count'
    )

    absent_count = ti.xcom_pull(
        task_ids='count_absent',
        key='absent_count'
    )

    total_students = present_count + absent_count

    with open('/tmp/attendance_report.txt', 'w') as f:
        f.write(f"Total Students = {total_students}\n")
        f.write(f"Present = {present_count}\n")
        f.write(f"Absent = {absent_count}\n")

    print("Attendance report generated successfully.")

with DAG(
    dag_id='attendance_report',
    start_date=datetime(2026, 6, 23),
    schedule=None,
    catchup=False
) as dag:

    create_attendance_task = PythonOperator(
        task_id='create_attendance',
        python_callable=create_attendance
    )

    count_present_task = PythonOperator(
        task_id='count_present',
        python_callable=count_present
    )

    count_absent_task = PythonOperator(
        task_id='count_absent',
        python_callable=count_absent
    )

    generate_summary_task = PythonOperator(
        task_id='generate_summary',
        python_callable=generate_summary
    )

    # Task Dependencies
    create_attendance_task >> \
    count_present_task >> \
    count_absent_task >> \
    generate_summary_task
```

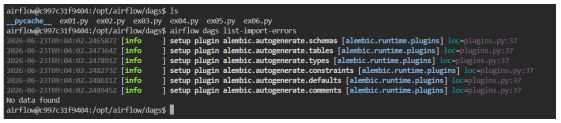


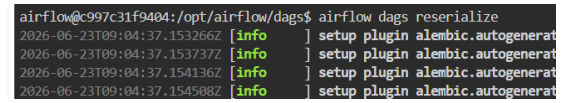
ex06.py

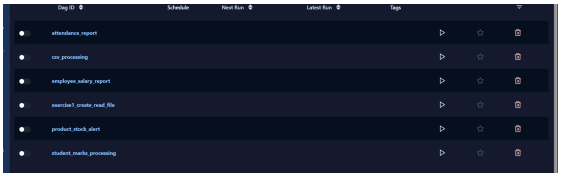
```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime
import csv

# Task 1: Create CSV file
def create_csv():
    with open('/tmp/sales.csv', 'w', newline='') as f:
        writer = csv.writer(f)
```

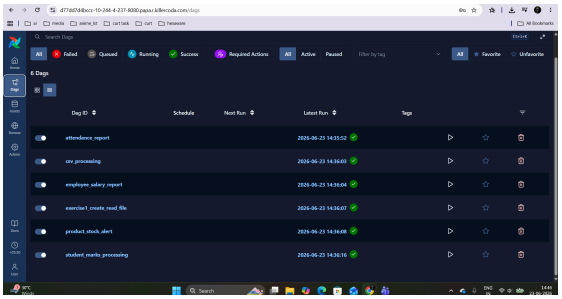
```
        writer.writerow(['product', 'quantity', 'price'])
        writer.writerow(['Laptop', 2, 70000])
        writer.writerow(['Mouse', 5, 500])
        writer.writerow(['Keyboard', 3, 1200])

    print("CSV file created successfully.")

# Task 2: Read CSV file
def read_csv_file():
    with open('/tmp/sales.csv', 'r') as f:
        reader = csv.reader(f)

        print("Sales Data:")
        for row in reader:
            print(row)

# Task 3: Calculate revenue
def calculate_revenue(ti):
    total_revenue = 0
    product_revenues = []

    with open('/tmp/sales.csv', 'r') as f:
        reader = csv.DictReader(f)

        for row in reader:
            revenue = int(row['quantity']) * int(row['price'])
            total_revenue += revenue

            product_revenues.append(
                f"{row['product']} = {revenue}"
            )

            print(f"{row['product']} = {revenue}")

    print(f"Total Revenue = {total_revenue}")

    ti.xcom_push(
        key='product_revenues',
        value=product_revenues
    )

    ti.xcom_push(
        key='total_revenue',
        value=total_revenue
    )

# Task 4: Create summary file
def create_summary(ti):
    product_revenues = ti.xcom_pull(
        task_ids='calculate_revenue',
        key='product_revenues'
    )

    total_revenue = ti.xcom_pull(
        task_ids='calculate_revenue',
        key='total_revenue'
    )

    with open('/tmp/summary.txt', 'w') as f:
        f.write("Revenue Summary\n")

        for item in product_revenues:
            f.write(item + '\n')

        f.write(f"Total Revenue = {total_revenue}")

    print("Summary file created successfully.")

with DAG(
    dag_id='csv_processing',
    start_date=datetime(2026, 6, 23),
    schedule=None,
    catchup=False
) as dag:

    create_csv_task = PythonOperator(
        task_id='create_csv',
        python_callable=create_csv
    )

    read_csv_task = PythonOperator(
        task_id='read_csv',
        python_callable=read_csv_file
    )

    calculate_revenue_task = PythonOperator(
        task_id='calculate_revenue',
        python_callable=calculate_revenue
    )

    create_summary_task = PythonOperator(
        task_id='create_summary',
        python_callable=create_summary
    )

    # Task Dependencies
    create_csv_task >> \
    read_csv_task >> \
    calculate_revenue_task >> \
    create_summary_task
```




airflow@c997c31f9404:/opt/airflow/dags$ ls
__pycache__ ex01.py ex02.py ex03.py ex04.py ex05.py ex06.py
airflow@c997c31f9404:/opt/airflow/dags$ airflow dags list-import-errors
2026-06-23T09:04:02.280567Z [info] setup plugin alembic.autogenerate.schemas [alembic.runtime.plugins] loc=plugins.py:37
2026-06-23T09:04:02.247364Z [info] setup plugin alembic.autogenerate.tables [alembic.runtime.plugins] loc=plugins.py:37
2026-06-23T09:04:02.247891Z [info] setup plugin alembic.autogenerate.types [alembic.runtime.plugins] loc=plugins.py:37
2026-06-23T09:04:02.248273Z [info] setup plugin alembic.autogenerate.constraints [alembic.runtime.plugins] loc=plugins.py:37
2026-06-23T09:04:02.248611Z [info] setup plugin alembic.autogenerate.defaults [alembic.runtime.plugins] loc=plugins.py:37
2026-06-23T09:04:02.248945Z [info] setup plugin alembic.autogenerate.comments [alembic.runtime.plugins] loc=plugins.py:37
No data found
airflow@c997c31f9404:/opt/airflow/dags$


airflow@c997c31f9404:/opt/airflow/dags$ airflow dags reserialize
2026-06-23T09:04:37.153266Z [info] setup plugin alembic.autogenerat
2026-06-23T09:04:37.153737Z [info] setup plugin alembic.autogenerat
2026-06-23T09:04:37.154136Z [info] setup plugin alembic.autogenerat
2026-06-23T09:04:37.154508Z [info] setup plugin alembic.autogenerat


Dag ID
Schedule
Next Run
Latest Run
Tags
attendance_report
csv_processing
employee_salary_report
exercise1_create_read_file
product_stock_alert
student_marks_processing


All
Failed
Queued
Running
Success
Required Actions
All
Active
Paused
All
Favorite
Unfavorite
6 Dags
Dag ID
Schedule
Next Run
Latest Run
Tags
attendance_report
2026-06-23 14:35:52
csv_processing
2026-06-23 14:36:03
employee_salary_report
2026-06-23 14:36:04
exercise1_create_read_file
2026-06-23 14:36:07
product_stock_alert
2026-06-23 14:36:08
student_marks_processing
2026-06-23 14:36:16